

A Practical Comparative Study of Scalable and Secure Backend Architectures for Resource-Constrained Web Applications

Shivanshu

BE-CSE

Chitkara University

Rajpura, India

shivanshu2330.be22@chitkara.edu.in

Shubham Goyal

BE-CSE

Chitkara University

Rajpura, India

shubham2364.be22@chitkara.edu.in

Animesh Chaudhri

BE-CSE

Chitkara University

Rajpura, India

animesh1266.be22@chitkara.edu.in

Shwetank Dohroo

BE-CSE

Chitkara University

Rajpura, India

shwetank2373.be22@chitkara.edu.in

Abstract— Today's web applications experience the challenge of supporting the ever-increasing number of users while providing sufficient security guarantees under resource constraints like budget and computation. It is still non-trivial to determine the best backend architecture, especially for small development teams, research centers, and startups. This paper provides an empirical evaluation of the popular backend architectures, including monolithic, microservices, and hybrid, with respect to scalability and security for resource-constrained web applications. From the experimental results, the monolithic architecture is seen to perform well under low workloads, while its performance degrades considerably under high concurrency. The microservices architecture supports the maximum throughput under high concurrency, with the cost of high memory usage. The hybrid architecture strikes the right trade-off between performance and security by centralizing the security functions at the gateway layer.

Keywords— Backend Architecture, Scalability, Security, Microservices, Resource-Constrained Systems

I. INTRODUCTION

The increased number of web-based services has put a strain on backend infrastructure, and developers are required to pick a backend architecture that scales with changing traffic while still providing security, within certain budgetary and hardware constraints. This is especially critical for small businesses such as startups and research labs, where over-engineering is costly and under-engineering causes system failures.

There are three backend architectures in current usage: monolithic, where all components of a backend are in a single process; microservices, where backend components are

deployed separately and communicate over a network; and hybrid, where there is a monolithic backend with an API gateway handling cross-cutting concerns such as authentication and rate limiting.

Other researchers have compared monolithic and microservices architecture in terms of throughput and resource usage, but there have been no studies conducted under the context of shared commodity hardware and low memory. This paper attempts to bridge the gap by designing an experiment to compare the three architectures in terms of response time, throughput, memory usage, and security under low, medium, and high load scenarios. The rest of this paper is telling this: Section 2 looks at other work that is similar. Section 3 talks about the methodology and experimental setup. Section 4 shows the experimental results. Section 5 talks about the limitations.

II. Related Work

[1] On a single machine, the monolith outperforms its microservice equivalent, and vertical scaling is more cost-efficient than horizontal scaling in the Azure cloud environment, which is relevant to web applications with tight computational budgets.

[2] The empirical concurrency tests revealed 6% higher throughput with monolithic architecture than with microservices, while no difference was found in both architectures in the load tests, demonstrating the significance of workload in choosing a particular architecture.

[3] The performance of Kubernetes, K3s, KubeEdge, and ioFog is measured in terms of their memory footprint and support for remote edge devices, which are specific metrics for these technologies.

[4] Currently, most cloud systems use containers as their foundation, and Kubernetes is the primary way of managing these containers. There are lighter alternatives like K3s and KubeEdge.

[5] Tests have shown that applications based on microservices architecture, in which a system is divided into several independent parts, are more efficient in terms of hardware usage, cost-effective, and increase productivity compared to traditional applications based on a monolithic architecture

[6] As for logging in and managing access to a system by users, JSON Web Tokens (JWT) is a popular solution. However, these tokens do not take into account how a user behaves. A possible solution is to enhance JSON Web Tokens by introducing a “behavior score.”

[7] If we take a look at four different examples of companies that went from microservices to a monolithic architecture, such as Prime Video, which saved a lot of money, the main reasons for switching from microservices to a monolithic architecture are cost, complexity, scalability, and performance. This is good advice for systems that don’t have a lot of resources.

[8] If we are looking at a comparison of monolithic and microservices architecture for small IoT devices and are coding in Go, Python, and C++, a monolithic architecture is better for small-scale apps because it uses less power, runs faster, and uses less memory.

[9] If we take a look at a survey of WebAssembly runtimes such as those found in browsers (Chrome, Firefox, Edge) and those that are stand-alone (Wasmtime, Wasmr, WasmEdge, Wasm3), the performance of the code is dependent on how the code is run and if it is interpreted, JIT compiled, or AOT compiled.

[10] If we are looking at eight different deep learning models for predicting solar power, sequence-based models such as LSTM and GRU are best for handling time-based data. The choice of a model is based on the trade-off between accuracy and computing cost.

[11] The use of a Docker framework based on Dynamic RESTful APIs and FastAPI is proposed to overcome the drawbacks of static APIs for IoT systems. The evaluation indicates high performance with a 3.11% error rate even under heavy loads and low latency despite changing conditions.

[12] Comparison of Docker and WebAssembly for containerization based on startup time, execution speed, memory size, portability, and security isolation indicates that although Wasm performs better for startup and binary sizes, Docker is a better choice for complex systems due to maturity.

[13] The use of Docker containers for microservices-based systems is shown to have a positive effect on scalability, portability, and deployment of distributed systems by decoupling microservices and managing dependencies in CI/CD pipelines.

[14] A survey of edge AI acceleration methods such as model compression (model pruning, quantization, knowledge distillation), NAS, and compiler-based approaches (TVM, TensorRT, and OpenVINO) indicates that a combination of all three leads to latency, memory, and power reductions without compromising the accuracy of the models.

III. METHODOLOGY

A. Application

The test application is split into three modules: a data processing endpoint that is heavy on writes and performs insertions and aggregations, a report endpoint that queries and returns data aggregations, and an authentication module for registration, login, and JWT issuance. The architecture of these three modules is kept identical for a fair comparison of the three approaches without any implementation bias. No new features are added in these modules.

B. Hardware and Software

The experiments were done on a personal laptop running Arch Linux. The laptop used was an ASUS TUF Gaming F15 FX507ZM with a 12th Gen Intel Core i7-12700H processor, 16 GB RAM, NVIDIA RTX 3060 Laptop GPU, and SSD storage. The same system was used for running the backend applications, databases, and load testing. Docker was used for the microservices setup and NGINX was used in the hybrid architecture setup. Testing was performed locally using k6 with different virtual user loads, and the average values from multiple runs were used in the final results

Software versions: Node.js 18.17.1, Express 4.18.2,

PostgreSQL 15.3, MongoDB 6.0.7, Docker 24.0.5, NGINX 1.24.0, PM2 5.3.0, k6 v0.45.0. PM2 was used for process management across all three setup to make sure consistency.

C. Architecture Details

Monolith.

A single Node.js process in which all three modules operate as Express routers within the same application. PostgreSQL is used for user and session data, while MongoDB is used for processed records and reports. PM2 has been added to all three of these configurations. This is because, during the initial testing phase, the application crashed due to high loads, necessitating manual restarts. PM2 has been added uniformly to all three configurations in order to maintain comparability.

Microservices.

Each of the three functional components—authentication, data processing, and reporting—runs as an independent Node.js application in a separate Docker container. All these services communicate with each other over a REST-based protocol over a Docker user-defined bridge network. During the initial testing phase, latency spikes of 5-15ms were observed between containers. Further investigation revealed that the Docker bridge network does a DNS resolution per call. Switching to a user-defined bridge network with static IP addresses resolved the problem.

Hybrid.

The same Node.js application is deployed as a monolith and is accessed through a reverse proxy using NGINX. The NGINX performs a JWT authentication using the “`ngx_http_auth_jwt_module`.” It is also responsible for rate limiting using “`limit_req_zone`” (100 requests per second per IP address; 429 is sent as a response for more requests). The

Node.js application layer is not responsible for any form of authentication since requests are already authenticated by the gateway layer. The other two architectures did not have a TLS layer because testing is done in a local network environment. This is a limitation in comparing the two architectures in terms of security and is discussed in Section

D. Load Testing

Load testing is done using k6 load testing tool and has three different virtual users (VU) levels: 50 VU (low), 200 VU (medium), and 500 VU (high). Each VU level is tested three times and then averaged. Each load level runs for 3 minutes at a constant VU count and has a 30-second ramp-up and ramp-down time. A k6 test script has been implemented for realistic workload using authentication and acquisition of JWT token, and then making five consecutive requests for data processing and two consecutive requests for reports.

The below-mentioned parameters have been measured: average response time, 95th percentile response time, RPS, RSS memory usage, CPU usage, and HTTP error rate.

CPU and memory usage information has been logged every 5 seconds using a custom monitoring script running on the application server.

IV. EXPERIMENTS AND RESULTS

A. Experiment setup

The experimental setup used in this study is available in the project repository. The repository contains the source code for monolithic, microservices, and hybrid architectures along with Docker Compose configuration files and load testing scripts. The setup can be reproduced by cloning the repository, installing Docker and Docker Compose, and running the command `docker compose up --build`. After starting the containers, the services can be accessed locally through localhost ports for testing and monitoring. Load testing was performed using the k6 framework on the running services

```
# Install Docker and Docker Compose (Arch Linux)
sudo pacman -S docker docker-compose
```

```
# Start Docker Service
Sudo systemctl start docker
```

```
#Enable Docker service on boot
sudo systemctl enable docker
```

```
#Run all services
Sudo docker compose up -build
```

After the setup is complete, open the following URLs in the browser

Dashboard <http://localhost:5173>

Architecture	Load (VUs)	Avg RT (ms)	p95 RT (ms)	RPS	Error (%)
Monolith	50	42	73	117	0.00
Microservices	50	63	101	80	0.00
Hybrid	50	55	86	91	0.00
Monolith	200	191	423	103	0.33
Microservices	200	129	207	154	0.00
Hybrid	200	146	241	137	0.00
Monolith	500	1347	3210	69	4.71
Microservices	500	188	318	265	0.19
Hybrid	500	258	503	194	0.51

Table 1. Response time (RT) and throughput at three load levels. Averages over three runs.

B. Response Time & Throughput

Table 1 presents response time and throughput results for all three architectures at all three load levels.

Values are averages over three runs.

At 50 VUs, all three architectures performed error-free. The monolith architecture recorded the minimum average response time, at 42 ms, because of the lack of network calls between services and gateway forwarding. The microservices architecture was 20 ms slower on average compared to the monolith architecture because of REST communication between Docker containers.

At 200 VUs, performance issues began to emerge in the monolith architecture. The average response time increased to 191 ms, p95 was 423 ms, and an error rate of 0.33% was recorded. This was because of a connection pool exhaustion error in PostgreSQL. The default connection pool size was set to 10, which was insufficient at 200 concurrent users making authentication requests. The connection pool was intentionally set to default to observe performance without manual tuning. Both microservices and hybrid architectures performed error-free at 200 VUs.

At 500 VUs, the monolith exhibited severe degradation: average response time reached 1347 ms, p95 exceeded 3 seconds, and the error rate was 4.71%. Errors were predominantly HTTP 503 responses rather than timeouts. The Node.js event loop was operating at 88–92% of a single CPU core throughout the test, consistent with the single-threaded execution model saturating under concurrent request queuing.

The microservices configuration sustained 188 ms average response time at 500 VUs with a 0.19% error rate, with errors attributable to shared MongoDB connection limits across three containers. The hybrid configuration averaged 258 ms at 500 VUs. This result was slower than microservices, likely due to NGINX and the Node.js process competing for CPU on the same machine; however, this cannot be confirmed without isolating NGINX onto separate hardware.

C. Memory and CPU

During the 500 VU test, the three microservices containers together consumed approximately 840–870 MB RSS. The monolith used approximately 410 MB and the hybrid configuration approximately 590 MB total (including NGINX).

During initial high-load microservices testing, the machine ran out of memory. Without container memory limits, the combined RSS of all containers and database processes approached 7 GB on the 8 GB machine, causing the OS to swap and rendering the system unusable. Memory limits of 512 MB per container were subsequently applied via Docker Compose. All results reported in Table 1 reflect this configuration.

D. Security Behavior

Three security aspects were analyzed: rejection of unauthorized JWTs, implementing rate limiting, and handling multiple authentication failures for the same IP address.

Within the monolith, the JWT authentication takes place via Express middleware that comes before each protected endpoint. Out of 200 JWTs sent, only those who were not valid or unauthorized received HTTP 401 error code. One major problem with monolithic architecture in terms of security is misconfiguration.

Regarding the microservices setup, each microservice has its own way of validating the token. During testing, a bug has been found within the data processing microservice that validates the JWT signature but neglects checking the exp claim, thus accepting expired tokens. This has been discovered after testing using a token about three hours old. This is one major pitfall associated with implementing microservices architecture, where the security features are implemented independently by each microservice leading to potential inconsistencies in how security is done within the system.

In the hybrid configuration, all authentication logic is handled exclusively by NGINX before requests reach the application. Invalid, expired, and missing tokens all returned HTTP 401, rate-limited requests returned HTTP 429. Centralizing this logic in a single gateway component eliminates the risk of inconsistent across services.

V. LIMITATIONS

A number of limitations must be kept in mind when evaluating the findings above. First, it must be noted that database isolation per service was not achieved within the microservices implementation, as advised by the microservices paradigm. All services had to work with one PostgreSQL database and one MongoDB database each with its own set of credentials

Next, the hybrid approach utilized TLS termination at the NGINX level, unlike the monolith and microservices approaches. This creates some inaccuracy in terms of security evaluation; however, it represents a realistic setting for a local network.

Next, the NGINX web server and the Node.js application were hosted on one computer in the hybrid setup. By having two applications run on the same computer, there will always be CPU contention problems that cannot arise when using dedicated computers. Poor performance for the hybrid setup under 500 VUs can, therefore, be linked to this disadvantage.

Next the test computers are common equipment in a college laboratory setting. While tests were run at non-peak hours during the evening, not all aspects such as the computer's background programs or network connectivity could be controlled

Lastly, there were no adjustments made to database tuning; the connection pool size, cache, and indexes were left at their default values. For instance, the issue of connection pool exhaustion in the monolithic architecture at 200 VUs could have been addressed by increasing the default pool size. This decision was made to ensure that the two architectures were compared based on their deployment without any optimizations.

VI. CONCLUSION

The current paper has experimentally analyzed the performance of monolithic, microservices, and hybrid backend architectures under certain load scenarios using commodity computers. Based on the obtained results, the following conclusions can be made.

Firstly, the monolithic architecture represents the most efficient architecture under low load scenarios, as there is no overhead involved in the interaction between different services. In particular, the average response time under 50 VUs equals 42 ms.

At the same time, the monolithic architecture turns out to be very vulnerable to heavy loads. Thus, the connection pools will run out when there are 200 VUs, and the single-threaded event loop will be overloaded by 500 VUs.

The microservices architecture recorded the highest performance during a high load with an RPS of 265 when using 500 virtual users. The microservices architecture uses twice as much memory as the monolithic architecture. The implementation of the JWT validation method in the microservices architecture created a security vulnerability which does not exist in the monolithic architecture.

The hybrid architecture provided a good compromise in terms of throughput at medium and high loads, zero errors at 200 VUs, and best-in-class security by offering centralized gateway-level authentication and rate limiting. The defect related to the JWT expiration in microservices could have been avoided by using the hybrid architecture as well. For resource-constrained teams in need of a path towards better scalability without the overhead of microservices complexity, the hybrid architecture is a valid alternative and is easier to operate.

As future work, these architectures should be tested on dedicated hardware per component, along with production-grade database configurations and database per service instances, in order to achieve more accurate results in a more realistic setting.

VII. References

- [1] G. Blinowski, A. Ojdowska and A. Przybyłek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.
- [2] O. Al-Debagy and P. Martinek, "A Comparative Review of Microservices and Monolithic Architectures," *IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI)*, Budapest, Hungary, 2018, pp. 000149–000154, doi: 10.1109/CINTI.2018.8928192.
- [3] I. Čilić, P. Krivić, I. Podnar Žarko, and M. Kušek, "Performance Evaluation of Container Orchestration Tools in Edge Computing Environments," *Sensors*, vol. 23, p. 4008, 2023, doi: 10.3390/s23084008.
- [4] R. Vaño, I. Lacalle, P. Sowiński, R. S-Julián, and C. E. Palau, "Cloud-Native Workload Orchestration at the Edge: A Deployment Review and Future Directions," *Sensors*, vol. 23, p. 2215, 2023, doi: 10.3390/s23042215.
- [5] F. Tapia et al., "From Monolithic Systems to Microservices: A Comparative Study of Performance," *Applied Sciences*, vol. 10, p. 5797, 2020, doi: 10.3390/app10175797.
- [6] A. Bucko, K. Vishi, B. Krasniqi, and B. Rexha, "Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History," *Computers*, vol. 12, p. 78, 2023, doi: 10.3390/computers12040078.
- [7] R. Su, X. Li, and D. Taibi, "From Microservice to Monolith: A Multivocal Literature Review," *Electronics*, vol. 13, p. 1452, 2024, doi: 10.3390/electronics13081452.
- [8] S. du Plessis and N. Correia, "A Comparative Study of Software Architectures in Constrained Device IoT Deployments," *IEEE International Conference on Internet of Things and Intelligence Systems (IoTIS)*, 2021.
- [9] M. Țălu, "A Comparative Study of WebAssembly Runtimes: Performance Metrics, Integration Challenges, Application Domains, and Security Features," *Archives of Advanced Engineering Science*, 2025.
- [10] M. Abdelsattar et al., "Comparative Analysis of Deep Learning Architectures in Solar Power Prediction," *Scientific Reports*, vol. 15, p. 31729, 2025.
- [11] E. Mabotha, N. E. Mabunda, and A. Ali, "A Dockerized Approach to Dynamic Endpoint Management for RESTful Application Programming Interfaces in Internet of Things Ecosystems," *Sensors*, vol. 25, p. 2993, 2025.
- [12] N. A. Nguyen, "Comparison of WebAssembly and Container Technologies," 2025.
- [13] M. M. Raikar et al., "Leveraging Docker Containers for Deployment of Web Applications in Microservices Architecture," *First International Conference for Women in Computing (InCoWoCo)*, IEEE, 2024.
- [14] K. Aruna and P. Gurunathan, "Enhancing Edge Environment Scalability: Leveraging Kubernetes for Container Orchestration and Optimization," *Concurrency and Computation: Practice and Experience*, vol. 36, p. e8303, 2024.
- [15] J. Sander et al., "On Accelerating Edge AI: Optimizing Resource-Constrained Environments," *arXiv preprint arXiv:2501.15014*, 2025.